

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Lab Task

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ikram

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

1. Introduction and Objective

This report documents a practical lab task on **Ansible**, an automation tool used to configure and manage remote machines. Ansible runs on a control node and connects to the machines it manages over SSH. The machines are listed in an **inventory file**, and the work to be done on them is described in a **playbook**. Because Ansible needs nothing but SSH and Python on the managed machines, it is widely used in DevOps for repeatable, consistent system configuration.

In this lab, the control node (user `samikhan`, host `osboxes`) is used to manage two Ubuntu machines named `m2` and `m3`. The student verifies the Ansible installation, tests SSH access to both machines, creates and checks an inventory file, tests connectivity with the Ansible ping module, and finally runs a playbook that installs Visual Studio Code on both machines.

Objectives

- Verify that Ansible is installed on the control node.
- Confirm SSH access to the managed machines `m2` (192.168.133.132) and `m3` (192.168.133.133).
- Create the inventory file `inventory.ini` and verify that Ansible reads it correctly.
- Test connectivity to both machines with the Ansible `ping` module.
- Write and run the playbook `install-vscode.yml` to install Visual Studio Code on both machines.

Lab Environment

Item	Details
Control node	samikhan@osboxes (working directory: ~/ansible-lab)
Managed node m2	192.168.133.132
Managed node m3	192.168.133.133
Login user on managed nodes	ansible
Managed node operating system	Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64)
Ansible version	ansible [core 2.18.1]
Inventory file	inventory.ini
Playbook	install-vscode.yml

2. Verifying the Ansible Installation

Before any automation is attempted, the student first confirms that Ansible is installed on the control node and checks which version and settings it is using. This avoids confusing installation problems with configuration problems later in the lab.

Commands Used

```
cd ~/ansible-lab
ansible --version
```

Command Explanation

- `cd ~/ansible-lab` — changes the current directory to the `ansible-lab` folder in the home directory (`~`). All files for this lab (inventory and playbook) are kept in this folder.

- `ansible --version` → prints the installed Ansible version together with its configuration details, such as the module search path, the Python version and the location of the executable.

What is being done. The student moves into the lab working directory and then asks Ansible to report its version. This is the first check that the tool is ready to use.

Result. The output shows `ansible [core 2.18.1]`, so Ansible is installed and working. It also shows that the executable is `/usr/bin/ansible`, that Python 3.13.3 is used, and that Jinja 3.1.5 is available. The line `config file = None` means that no `ansible.cfg` file is being used, so Ansible runs with its default settings; for this reason the inventory is passed explicitly with the `-i` option in later commands.

```
samikhan@osboxes:~$ cd ~/ansible-lab
samikhan@osboxes:~/ansible-lab$ ansible --version
ansible [core 2.18.1]
  config file = None
  configured module search path = ['/home/samikhan/.ansible/plugins/modules', '/
usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/samikhan/.ansible/collections:/usr/share/a
nsible/collections
  executable location = /usr/bin/ansible
  python version = 3.13.3 (main, Jan  8 2026, 12:03:54) [GCC 14.2.0] (/usr/bin/p
ython3)
  jinja version = 3.1.5
  libyaml = True
```

Figure 1: *Checking the Ansible installation with the ansible --version command.*

3. Testing SSH Access to the Managed Nodes

Ansible reaches its managed machines over SSH, so the student first logs in to each machine manually. In real-world DevOps work, this quick manual test confirms the IP address, the username and the password before they are written into an inventory. It also makes SSH save each machine's host key, so Ansible does not stop later to ask for confirmation.

3.1 Connecting to m2 (192.168.133.132)

Command Used

```
ssh ansible@192.168.133.132
```

Command Explanation

- `ssh` → opens a secure remote login session on another computer.
- `ansible@192.168.133.132` → the format is `username@address`: it logs in as the user `ansible` on the machine with IP address 192.168.133.132 (`m2`).

What is being done. The student connects from the control node to `m2` for the first time. Because the machine is not yet known to SSH, the client displays its ED25519 key fingerprint and asks whether to continue. The student answers `yes`, which stores the key, and then enters the password of the `ansible` user.

Result. SSH reports that the host was "permanently added" to the list of known hosts, accepts the password, and shows the welcome banner of Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64). The login to `m2` is successful. The banner continues below the visible part of the screenshot.

```
  libyaml = True
samikhan@osboxes:~/ansible-lab$ ssh ansible@192.168.133.132
The authenticity of host '192.168.133.132 (192.168.133.132)' can't be establishe
d.
ED25519 key fingerprint is SHA256:QxHXhu+XPJBVpE45mpHqAsE+bXTG+CeJGXrv1SyidFY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.133.132' (ED25519) to the list of known host
s.
ansible@192.168.133.132's password:
Welcome to Ubuntu 25.04 (GNU/Linux 6.14.0-37-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

0 updates can be applied immediately.


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
```

Figure 2: *Establishing the first SSH connection to m2 (192.168.133.132) as the ansible user.*

3.2 Connecting to m3 (192.168.133.133)

Commands Used

```
exit
ssh ansible@192.168.133.133
```

Command Explanation

- `exit` → closes the current SSH session and returns to the control node. Here it ends the session on `m2`; the output "Connection to 192.168.133.132 closed." confirms this.
- `ssh ansible@192.168.133.133` → logs in as the user `ansible` on the second managed machine, `m3`.

What is being done. The student leaves the `m2` session and repeats the same test for `m3`. Again SSH asks for confirmation, the student answers `yes`, and the password is entered.

Result. The new host is added to the list of known hosts and the Ubuntu 25.04 welcome banner appears, so the login to `m3` works. SSH also prints that the key for this address is already known under another name in `~/.ssh/known_hosts` (line 7); the fingerprint shown is the same as the one shown for `m2`.

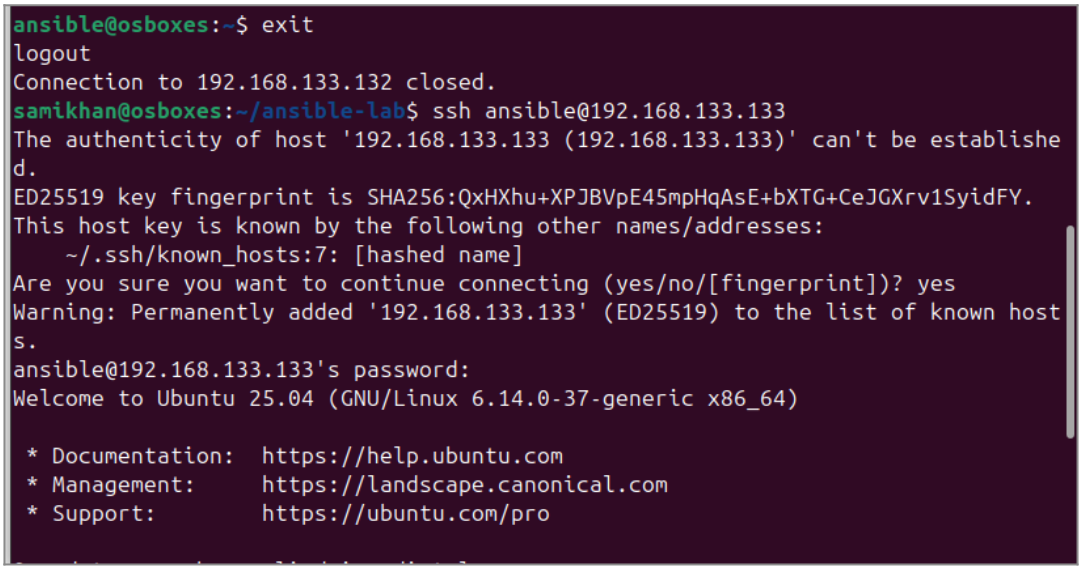


Figure 3: *Closing the m2 session and connecting to m3 (192.168.133.133) over SSH.*

Continuation. No new command is entered in the next screenshot. It shows the rest of the output of the same login: the remaining part of the welcome message and, at the bottom, the prompt `ansible@osboxes:~$`. This prompt confirms that the student is now logged in to `m3` as the user `ansible`.

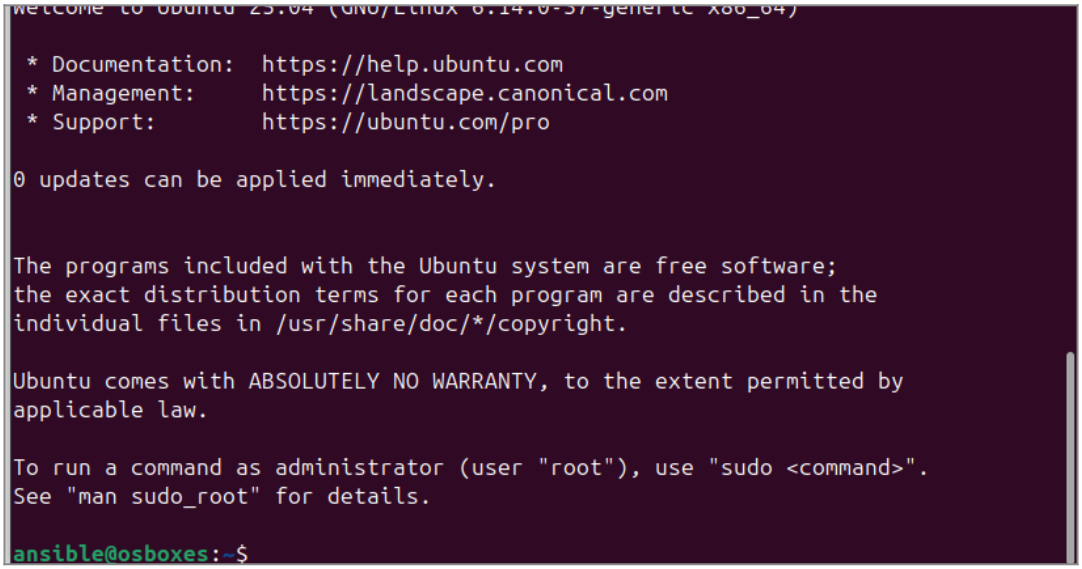


Figure 4: *Remainder of the m3 login output, ending at the ansible user prompt.*

4. Creating the Inventory File

An inventory file tells Ansible which machines it may manage and how to connect to them. In this step the student writes an inventory that groups `m2` and `m3` and stores their connection details.

Command Used

```
nano inventory.ini
```

Command Explanation

- `nano` → opens the nano text editor in the terminal.

- `inventory.ini` → the file to create or edit. It is stored in the `ansible-lab` directory.

What is being done. The student writes the inventory in nano. The file defines a group called `servers` with two hosts, and a second section that sets variables for every host in that group:

```
[servers]
m2 ansible_host=192.168.133.132
m3 ansible_host=192.168.133.133

[servers:vars]
ansible_user=ansible
ansible_password=ansible123
ansible_become_password=ansible123
```

- `[servers]` → the group name. `m2` and `m3` are short names for the hosts, and `ansible_host` gives the real IP address Ansible must connect to.
- `[servers:vars]` → variables that apply to every host in the `servers` group.
- `ansible_user` → the SSH username (`ansible`).
- `ansible_password` → the SSH password used to log in.
- `ansible_become_password` → the password used when Ansible needs administrator (sudo) rights.

Result. The nano window shows the file `inventory.ini` with the contents above. The asterisk after the file name in the title bar means the latest changes have not yet been saved; the bottom lines show the nano shortcuts, such as `^O` (Write Out) to save and `^X` (Exit) to close the editor. Note that keeping plain-text passwords in an inventory is acceptable in a closed lab, but in real projects SSH keys or Ansible Vault are normally used instead.

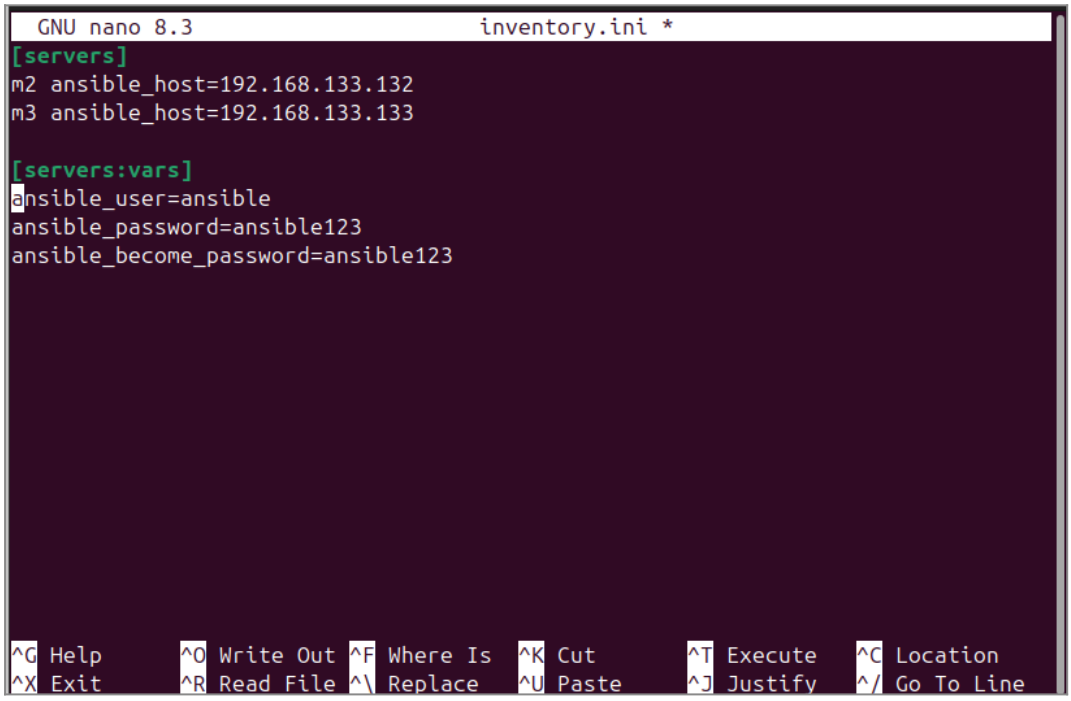


Figure 5: *Writing the inventory file inventory.ini in the nano editor.*

4.1 Saving and Checking the File

Commands Used

```
exit
nano inventory.ini
cat inventory.ini
```

Command Explanation

- `exit` → ends the SSH session on `m3` ("Connection to 192.168.133.133 closed.") and returns to the control node prompt `samikhan@osboxes:~/ansible-lab$`.
- `nano inventory.ini` → opens the inventory file in the editor, where it is saved and closed.
- `cat inventory.ini` → prints the content of the file on the screen so it can be checked.

What is being done. After leaving the `m3` session, the student returns to the editor to save the inventory file, and then prints it with `cat` to confirm that the saved file has the intended content.

Result. The `cat` output shows the `[servers]` group with `m2` (192.168.133.132) and `m3` (192.168.133.133) and the `[servers:vars]` section with `ansible_user`, `ansible_password` and `ansible_become_password`. This matches what was typed in nano, so the file was saved correctly.



Figure 6: *Leaving the m3 session and confirming the saved inventory file with cat.*

5. Verifying the Inventory

Having a file on disk does not prove that Ansible understands it. The student therefore asks Ansible itself to read the inventory and show how it was interpreted.

5.1 Displaying the Parsed Inventory

Command Used

```
ansible-inventory -i inventory.ini --list
```

Command Explanation

- `ansible-inventory` → a tool that shows how Ansible reads an inventory.
- `-i inventory.ini` → selects the inventory file to use.
- `--list` → prints all groups, hosts and variables in JSON format.

What is being done. The student asks Ansible to print the inventory it has loaded, in order to confirm that the hosts and variables were read as intended.

Result. The JSON output lists `m2` and `m3` under `_meta` → `hostvars`. Each host has its own `ansible_host` address (192.168.133.132 and 192.168.133.133) and also carries `ansible_user`, `ansible_password` and `ansible_become_password`, which it inherited from the `[servers:vars]` section. This shows that the inventory was parsed correctly. The output continues below the visible part of the screenshot (the `all` → `children` section begins at the bottom).



Figure 7: *Viewing the parsed inventory with ansible-inventory --list.*

5.2 Listing the Managed Hosts

Command Used

```
ansible all -i inventory.ini --list-hosts
```

Command Explanation

- `ansible` → the Ansible command-line tool for running tasks on managed hosts.
- `all` → the host pattern that selects every host in the inventory.
- `-i inventory.ini` → the inventory file to use.
- `--list-hosts` → only lists the hosts that match the pattern. It does not connect to them or run anything.

What is being done. The student checks which hosts Ansible would target when the pattern `all` is used.

Result. The output reads `hosts (2):` followed by `m2` and `m3`. Ansible therefore sees exactly the two managed machines defined in the inventory.

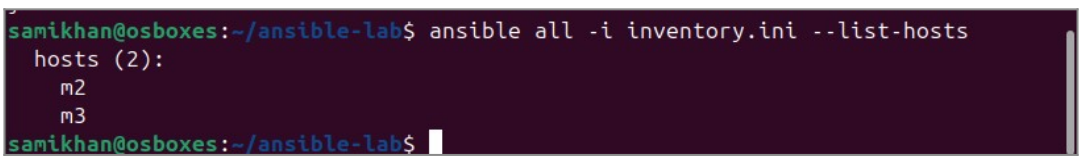


Figure 8: *Listing the hosts in the inventory with the --list-hosts option.*

6. Testing Connectivity with the Ansible Ping Module

The next check is the most important one before running a playbook: it confirms that Ansible can really log in to every managed machine and run its modules there.

Command Used

```
ansible all -i inventory.ini -m ping
```

Command Explanation

- `ansible all` → run on every host in the inventory.
- `-i inventory.ini` → use the inventory file created earlier.
- `-m ping` → use the Ansible `ping` module. This is not a network ICMP ping: it logs in over SSH, checks that Python can be used on the remote machine, and replies with `pong` if everything works.

What is being done. The student tests the whole Ansible connection chain (inventory, SSH login, credentials and Python) for both managed machines at once.

Result. Both `m3` and `m2` return `SUCCESS` with `"ping": "pong"` and `"changed": false`, which means the machines were reached and nothing on them was modified. The messages marked `[WARNING]` only say that Ansible automatically discovered the Python interpreter `/usr/bin/python3.13` on each host; they are informational and do not stop the command. The command prompt returns at the bottom of the output.

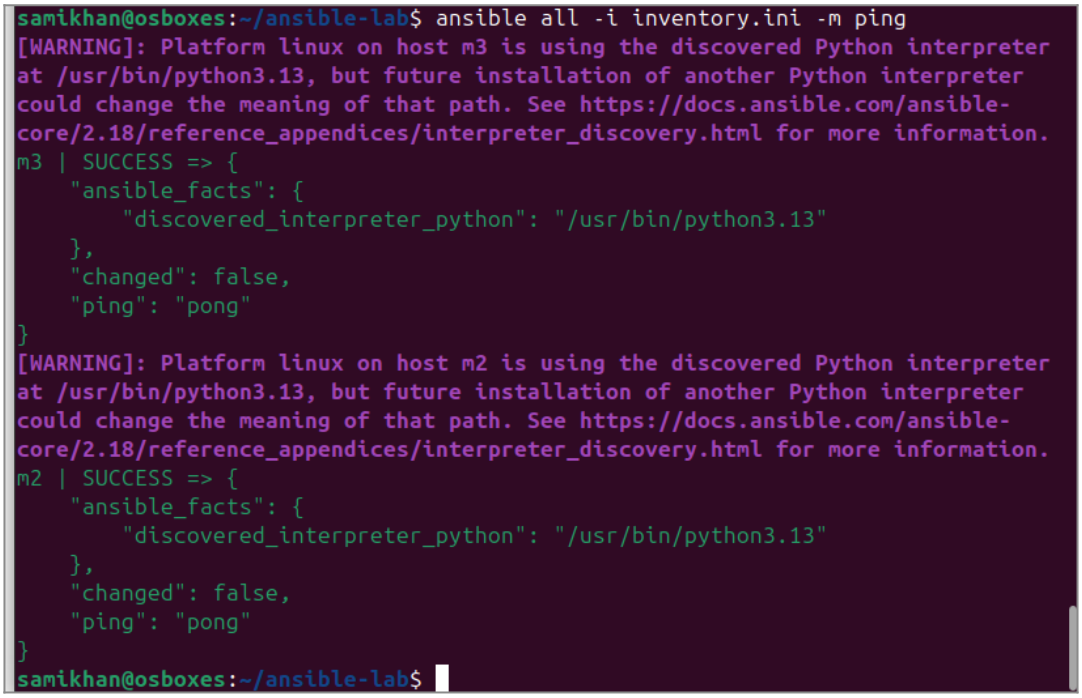


Figure 9: *Testing connectivity to m2 and m3 with the Ansible ping module.*

7. Writing the Playbook

A playbook describes, in YAML, the tasks Ansible must carry out on selected hosts. In real-world DevOps work, a playbook replaces a list of manual installation steps with one file that can be run again and again with the same result. Here the student writes the playbook `install-vscode.yml`, which installs Visual Studio Code on the hosts in the `servers` group.

File Being Edited

```
install-vscode.yml (in GNU nano 8.3)
```

Playbook Content (visible portion)

```
---
- name: Install Visual Studio Code on M2 and M3
  hosts: servers
  become: true

  tasks:
    - name: Install required packages
      ansible.builtin.apt:
        name:
          - wget
          - gpg
          - apt-transport-https
        state: present
        update_cache: true

    - name: Download Microsoft signing key
      ansible.builtin.get_url:
        url: https://packages.microsoft.com/keys/microsoft.asc
        dest: /tmp/microsoft.asc
        mode: '0644'
```

Explanation of the Playbook

- `name: Install Visual Studio Code on M2 and M3` – a readable title for the play.
- `hosts: servers` – the play runs on every host in the `servers` group of the inventory (`m2` and `m3`).
- `become: true` – tasks run with administrator (sudo) rights, using the `ansible_become_password` from the inventory.
- `tasks:` – the list of steps that Ansible performs in order.
- `ansible.builtin.apt` (task "Install required packages") – installs the packages `wget`, `gpg` and `apt-transport-https`; `state: present` makes sure they are installed and `update_cache: true` refreshes the package list first.
- `ansible.builtin.get_url` (task "Download Microsoft signing key") – downloads the key from `https://packages.microsoft.com/keys/microsoft.asc` to `/tmp/microsoft.asc` with file permissions `0644`.

What is being done. The student is typing the playbook in nano. The screenshot shows only the first part of the file; the remaining tasks ("Convert Microsoft signing key", "Add VS Code repository", "Update apt cache" and "Install Visual Studio Code") are not visible here, but they appear in the output when the playbook is executed in Section 9.

Result. The editor shows the opening of the playbook with the two tasks above. The asterisk in the title bar shows that the file has not been saved yet.

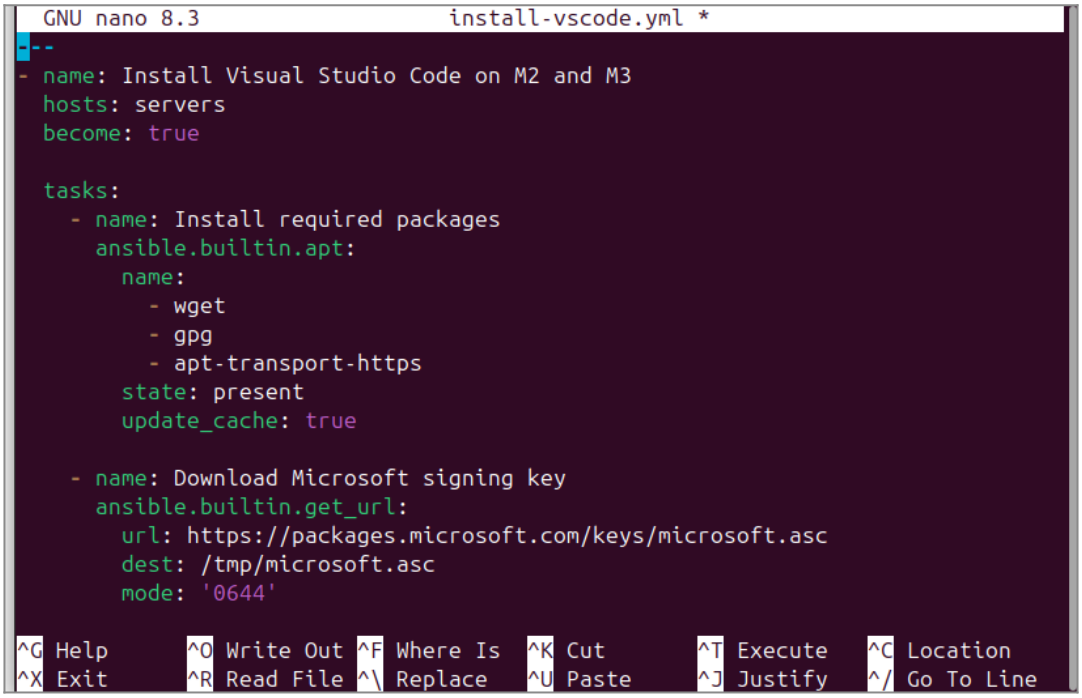


Figure 10: *Writing the install-vscode.yml playbook in the nano editor.*

8. Network Check and Package Preparation

Before running the playbook, the student checks network reachability of both machines with the standard `ping` command and updates the package lists with `apt`. The packages named in these commands are the same ones used by the playbook, namely `wget`, `gpg` and `apt-transport-https`.

8.1 Ping Test and Package Index Update

Commands Used

```
ping -c 4 192.168.133.132
ping -c 4 192.168.133.133
sudo apt update
sudo apt install -y wget gpg apt-transport-https
```

Command Explanation

- `ping` — sends small network test packets (ICMP echo requests) to a machine and waits for replies.
- `-c 4` — stop after sending 4 packets.
- `192.168.133.132` and `192.168.133.133` — the IP addresses of `m2` and `m3`.
- `sudo` — runs the command with administrator rights.
- `apt update` — refreshes the list of available packages from the repositories.
- `apt install -y wget gpg apt-transport-https` — installs the three packages; `-y` answers "yes" automatically to the confirmation question.

What is being done. The student tests whether both machines answer on the network and then starts preparing the package system. The commands were entered as pairs, so the second command of each pair is echoed on the line after the first.

Result. For `192.168.133.132` (`m2`), 4 packets were transmitted and 4 received with 0% packet loss; the round-trip time is about 0.5–0.8 ms (average 0.608 ms). For `192.168.133.133` (`m3`), the ping was interrupted with `Ctrl+C`

(shown as `^C`) and the statistics report 3 packets transmitted, 0 received and **100% packet loss**, so `m3` did not answer at that moment. This is relevant to the playbook run in Section 9. The `sudo apt update` command then asks for the password of `samikhan` and starts fetching package lists (the first lines are `Hit:1` and `Hit:2` for the Ubuntu security and archive repositories).

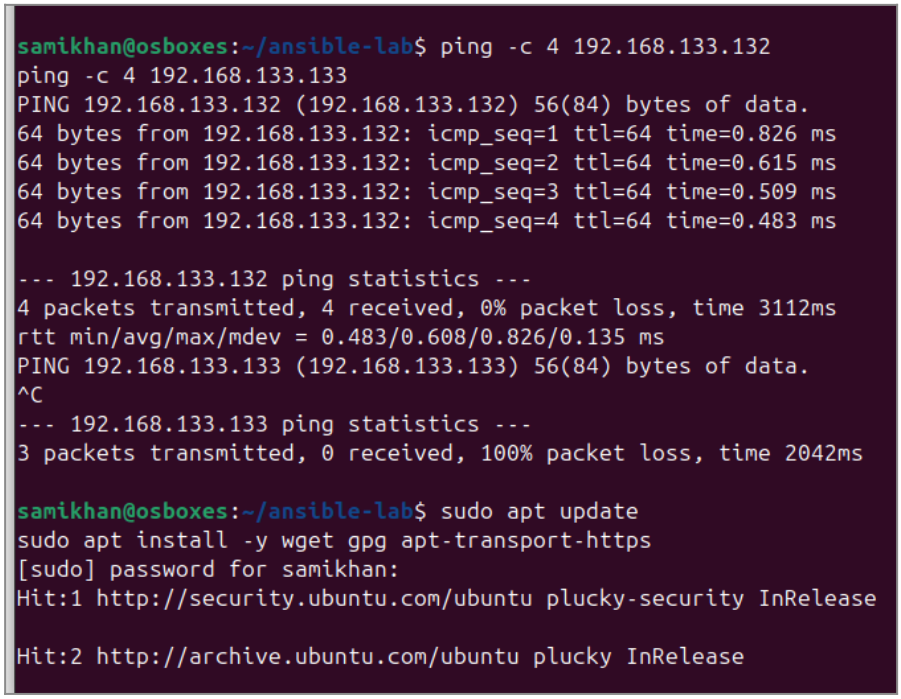


Figure 11: *Ping test of both machines followed by the start of sudo apt update.*

8.2 Package Lists and Installation of code

Commands Used

```
sudo apt update
sudo apt install -y code
```

Command Explanation

- `sudo apt update` → refreshes the package lists, as before.
- `sudo apt install -y code` → installs the package named `code`, which is Visual Studio Code. `-y` confirms the installation automatically.

What is being done. This screenshot starts again from the `sudo apt update` command, so it overlaps with the previous one and shows the rest of the output. The student updates the package lists and then installs the `code` package.

Result. The repositories Ubuntu security, archive, updates, backports and Docker are checked (`Hit`), and the Microsoft repository `packages.microsoft.com/repos/code stable` is fetched (`Get`). Apt reports "Fetched 34.6 kB in 1s (32.8 kB/s)" and "All packages are up to date". It also notes that some earlier-installed packages (`docker-ce-rootless-extras`, `libslirp0`, `slirp4netns`) are no longer required and can be removed with `sudo apt autoremove`. The last visible line is `Installing: code`; the screenshot ends there, so the completion of the installation is not shown.

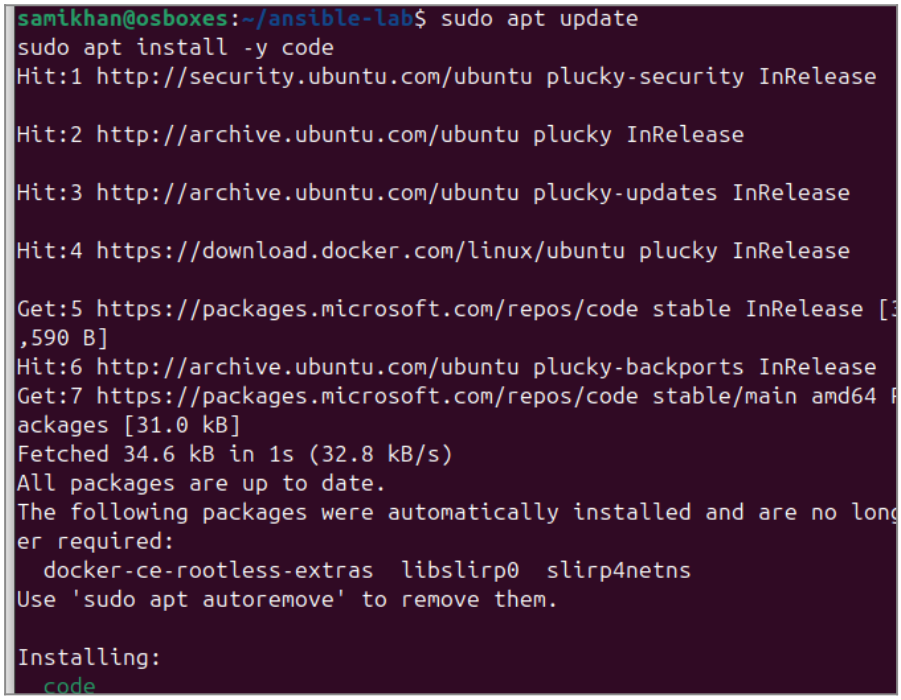


Figure 12: *Updating the package lists and starting the installation of the code package.*

9. Running the Playbook

With the inventory and the playbook ready, the student now runs the playbook to install Visual Studio Code on both managed machines.

Command Used

```
ansible-playbook -i inventory.ini install-vscode.yml
```

Command Explanation

- `ansible-playbook` → runs a playbook (a YAML file of tasks) instead of a single module.
- `-i inventory.ini` → the inventory that defines the target hosts.
- `install-vscode.yml` → the playbook to run.

What is being done. The student executes the playbook against the `servers` group. Ansible prints one `TASK` block for each task and reports the result for each host. The first line of the command is partly cut off at the top of the first screenshot, but the end of the file name `install-vscode.yml` is visible.

9.1 First Run: m3 Unreachable

Result. The play `Install Visual Studio Code on M2 and M3` starts and the first task, `Gathering Facts`, succeeds for `m2` (`ok: [m2]`; the interpreter warning is the same informational message as before). For `m3`, however, the output shows an **error**: `fatal: [m3]: UNREACHABLE!` with the message "Failed to connect to the host via ssh", followed by `ssh: connect to host 192.168.133.133 …` (the rest of the line is cut off). This means that Ansible could not open an SSH connection to `m3` at that time. It agrees with the failed ping to `m3` in Section 8.1. The report does not show how the connection to `m3` was restored, but the later screenshots show `m3` working.

```
nstall-vscode.yml

PLAY [Install Visual Studio Code on M2 and M3] ********************
*************

TASK [Gathering Facts] *********************************************
*************
[WARNING]: Platform linux on host m2 is using the discovered Python
 interpreter
at /usr/bin/python3.13, but future installation of another Python i
nterpreter
could change the meaning of that path. See https://docs.ansible.com
/ansible-
core/2.18/reference_appendices/interpreter_discovery.html for more
information.
ok: [m2]
fatal: [m3]: UNREACHABLE! => {"changed": false, "msg": "Failed to c
onnect to the host via ssh: ssh: connect to host 192.168.133.133 po
```

Figure 13: *Start of the playbook run: Gathering Facts succeeds on m2 while m3 is reported unreachable.*

Result (task progress on m2). Because `m3` is unreachable, only `m2` appears in the next tasks. Each of the tasks `Install required packages`, `Download Microsoft signing key`, `Convert Microsoft signing key`, `Add VS Code repository` and `Update apt cache` is reported as `changed: [m2]`, which means Ansible made a change on `m2` for each of them. The last task, `Install Visual Studio Code`, is listed at the bottom of the screenshot and its result is not visible there.

```
TASK [Install required packages] ***********************************
*************
changed: [m2]

TASK [Download Microsoft signing key] ******************************
*************
changed: [m2]

TASK [Convert Microsoft signing key] *******************************
*************
changed: [m2]

TASK [Add VS Code repository] **************************************
*************
changed: [m2]

TASK [Update apt cache] ********************************************
*************
changed: [m2]

TASK [Install Visual Studio Code] **********************************
*************
```

Figure 14: *Task-by-task progress of the playbook on m2.*

9.2 Final Run Summary

Result. In this screenshot, the task `Install Visual Studio Code` shows `ok: [m2]` and `changed: [m3]`, and the `PLAY RECAP` summarises the run. Both `m2` and `m3` have `ok=7`, `unreachable=0`, `failed=0`, `skipped=0`, `rescued=0` and `ignored=0`. The `changed` counts are 1 for `m2` and 6 for `m3`. In an Ansible recap, `ok` counts all steps that completed (including those that changed something) and `changed` counts the steps that modified the machine. For `m3`, all six tasks made a change, and Visual Studio Code was installed on it by this run. For `m2`, only one step changed anything, which suggests that most of the work had already been done by the earlier run in Section 9.1. Since `m3` is no longer unreachable here, this recap appears to belong to a later run after `m3` could be reached again.



Figure 15: *Final task result and PLAY RECAP showing no failed or unreachable hosts.*

10. Final Connectivity Verification

As a last check, the student repeats the Ansible ping test to confirm that both managed machines can still be reached after the playbook has been run.

Command Used

```
ansible all -i inventory.ini -m ping
```

Command Explanation

This is the same command used in Section 6. `ansible all` selects every host in the inventory, `-i inventory.ini` supplies the inventory file, and `-m ping` runs the Ansible ping module, which tests the SSH login and Python on each host.

What is being done. The student runs the connectivity test a second time, now at the end of the lab. This is important because `m3` was reported as unreachable earlier in the playbook run.

Result. Both `m3` and `m2` return `SUCCESS` with `"ping": "pong"` and `"changed": false`. The `[WARNING]` lines are again only the informational Python interpreter discovery message. Both managed machines are therefore reachable by Ansible.

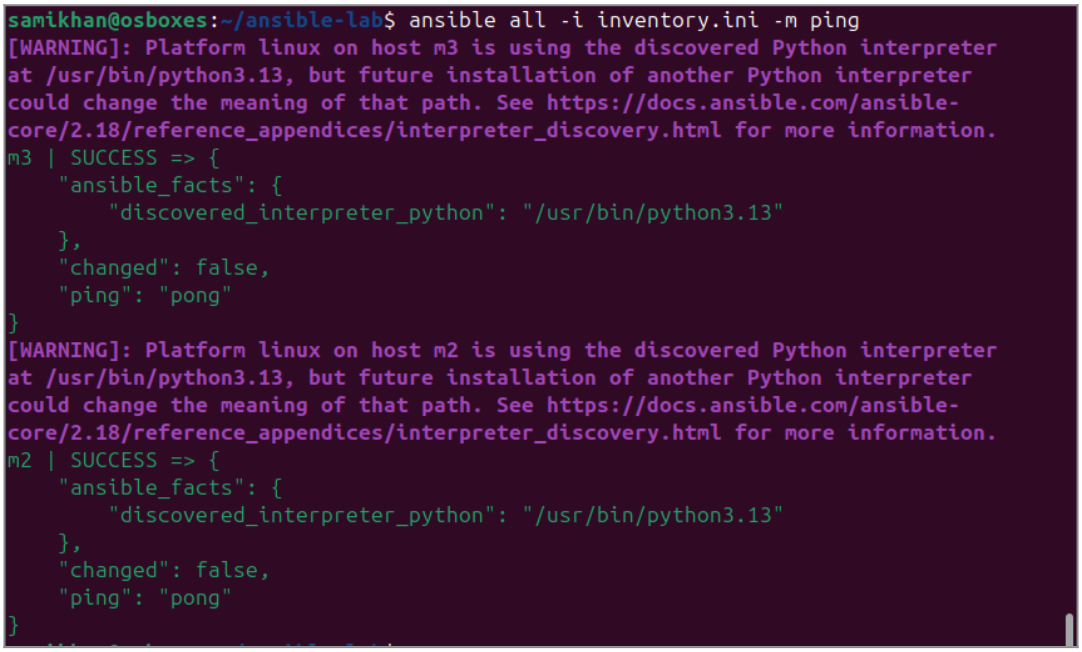


Figure 16: *Final Ansible ping test confirming that m2 and m3 are both reachable.*

11. Conclusion

In this lab, Ansible 2.18.1 was verified on the control node and SSH access to the managed machines `m2` (192.168.133.132) and `m3` (192.168.133.133) was tested. An inventory file, `inventory.ini`, was created, checked with `ansible-inventory` and `--list-hosts`, and used for the Ansible ping test, which succeeded on both machines. The playbook `install-vscode.yml` was then written and executed. In the first run `m3` was reported as unreachable, which matched a failed network ping; the final recap shows `failed=0` and `unreachable=0` for both machines, with the Visual Studio Code installation task reported as `changed` on `m3` and `ok` on `m2`. A final Ansible ping confirmed that both machines respond. The lab shows how a single inventory and playbook can configure several machines in the same way.